Adversarial Search (Games)

Boris Velichkov

Exercises by Week

1. Introduction
2. **Uninformed *(Blind)* Search**
3. **Informed *(Heuristic)* Search**
4. **Constraint Satisfaction Problems**
5. **Genetic Algorithms**
6. **Games**
7. Introduction to Machine Learning
8. ***k*-Nearest Neighbors**
9. **Naïve Bayes Classifier**
10. **Decision Tree**
11. ***k*Means**
12. **Neural Networks**
13. *Additional Topics, Questions and Homeworks' Presentations*
14. *Additional Topics, Questions and Homeworks' Presentations*
15. *Additional Topics, Questions and Homeworks' Presentations*

Games vs. Search Problems

- **"Unpredictable" opponent** → specifying a move for every possible opponent reply

- **Time limits** → unlikely to find goal, must approximate

Types of Games

	deterministic	chance
perfect information	chess, checkers, go, othello	backgammon monopoly
imperfect information	battleships, blind tictactoe	bridge, poker, scrabble nuclear war

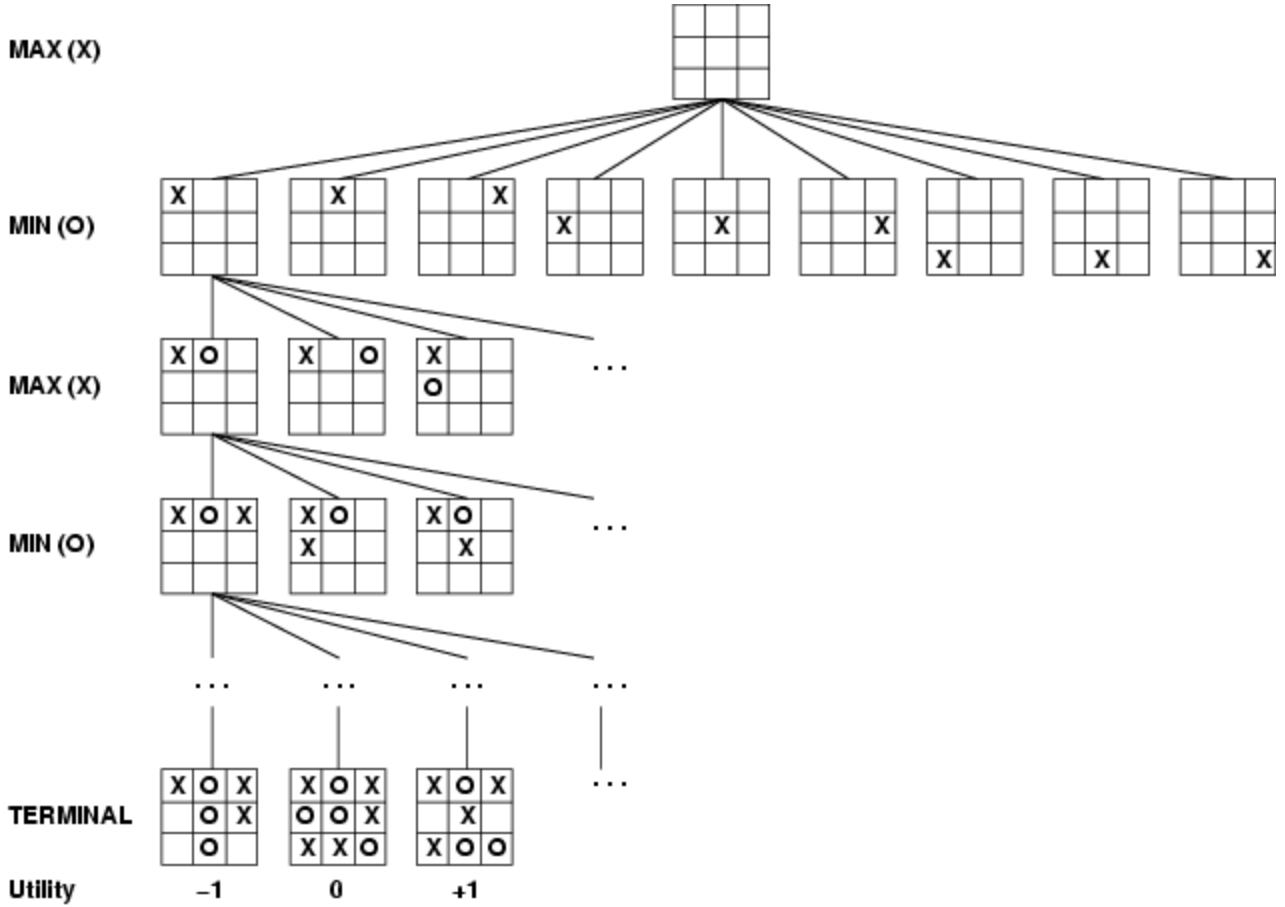
What about complete and incomplete information?

- An important subset of sequential games consists of games of perfect information. A game is one of perfect information if all players know the moves previously made by all other players.
- Complete information requires that every player know the strategies and payoffs available to the other players but not necessarily the actions taken.

Types of Games

- Cooperative / Non-cooperative
- Symmetric / Asymmetric
- Zero-sum / Non-zero-sum
- Simultaneous / Sequential
- Perfect information and imperfect information
- Combinatorial games
- Infinitely long games
- Discrete and continuous games
- Differential games
- Evolutionary game theory
- Stochastic outcomes (and relation to other fields)
- Metagames
- Pooling games
- Mean field game theory

Game tree (2-player, deterministic, turns)

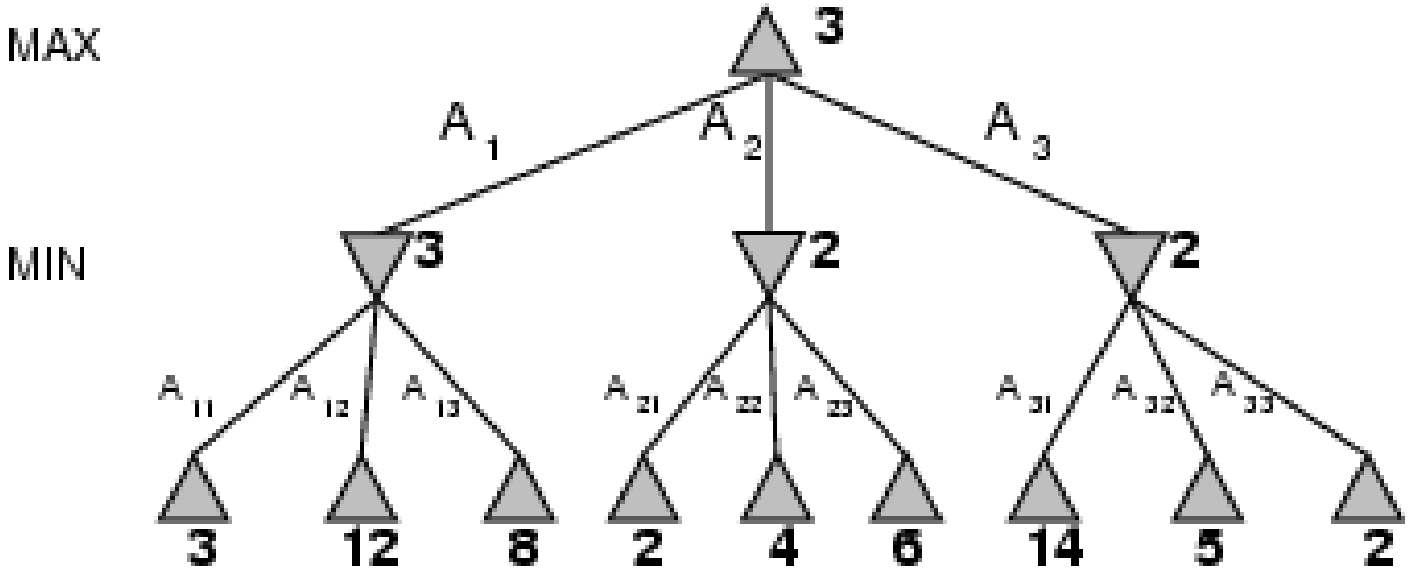


Minimax

Perfect play for deterministic (and perfect-information) games

Idea: choose move to position with highest minimax value
= best achievable payoff against best play

E.g., 2-ply game:



Minimax algorithm

```
function MINIMAX-DECISION(state) returns an action
    v ← MAX-VALUE(state)
    return the action in SUCCESSORS(state) with value v

function MAX-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do
        v ← MAX(v, MIN-VALUE(s))
    return v

function MIN-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← ∞
    for a, s in SUCCESSORS(state) do
        v ← MIN(v, MAX-VALUE(s))
    return v
```

Minimax algorithm

```
function MINIMAX-DECISION(state) returns an action
    inputs: state, current state in game

    return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))

---

function MAX-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do v ← MAX(v, MIN-VALUE(s))
    return v

---

function MIN-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← ∞
    for a, s in SUCCESSORS(state) do v ← MIN(v, MAX-VALUE(s))
    return v
```

Properties of minimax

Complete? Yes, if tree is finite (chess has specific rules for this)

NB: A finite strategy can exist even in an infinite tree!

Optimal? Yes, against an optimal opponent. Otherwise??

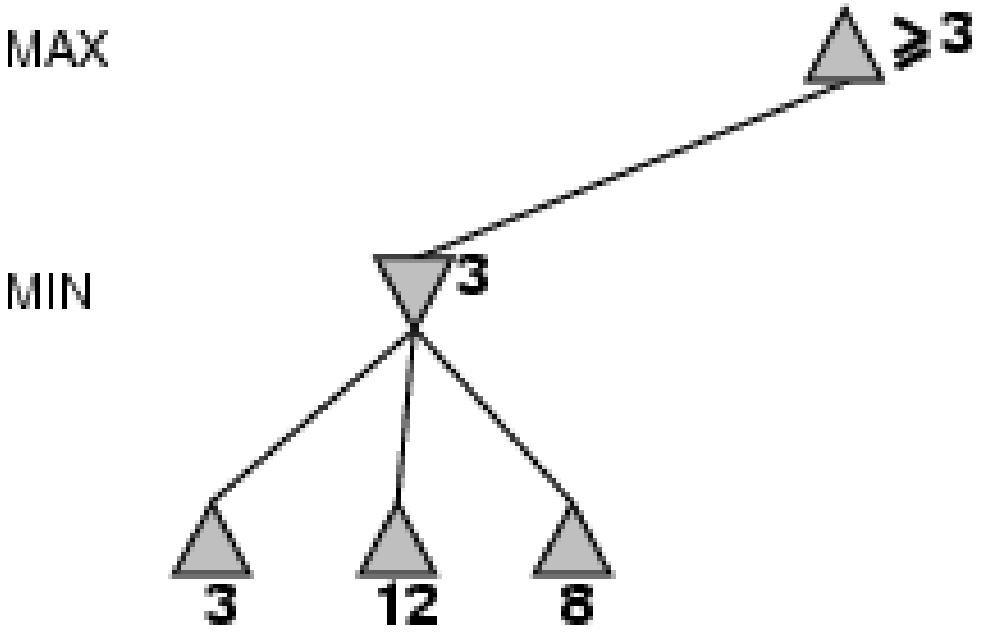
Time complexity? $O(b^m)$

Space complexity? O(bm) (depth-first exploration)

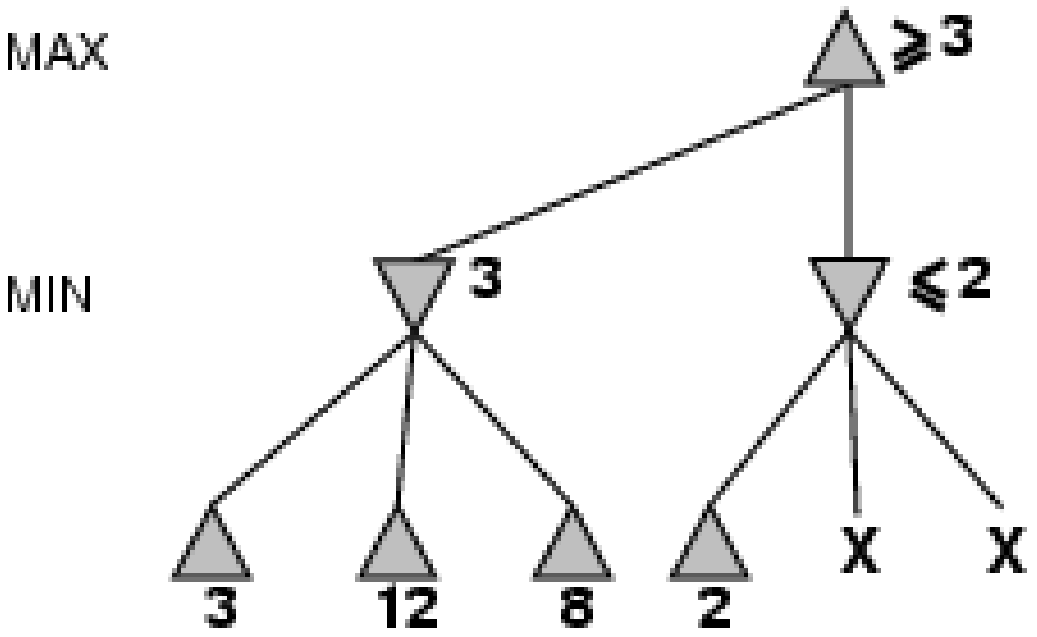
For chess, b ≈ 35, m ≈100 for "reasonable" games
→ exact solution completely infeasible

But do we need to explore every path?

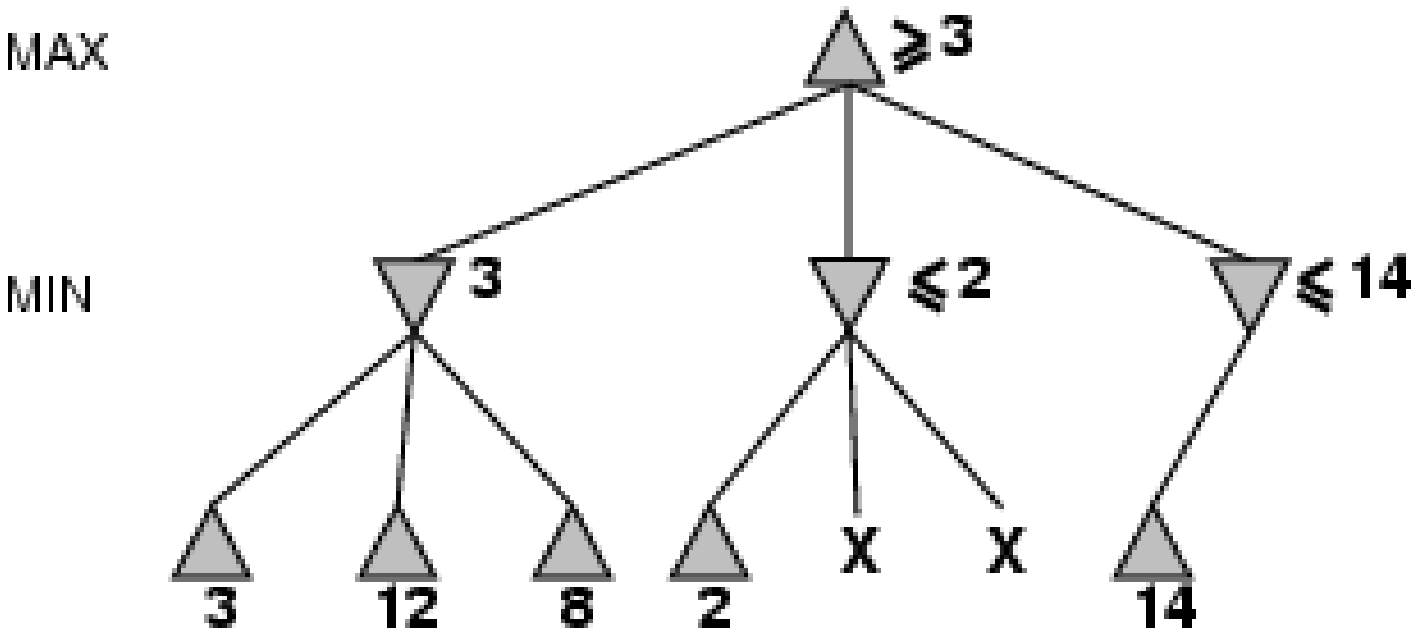
α-β pruning example



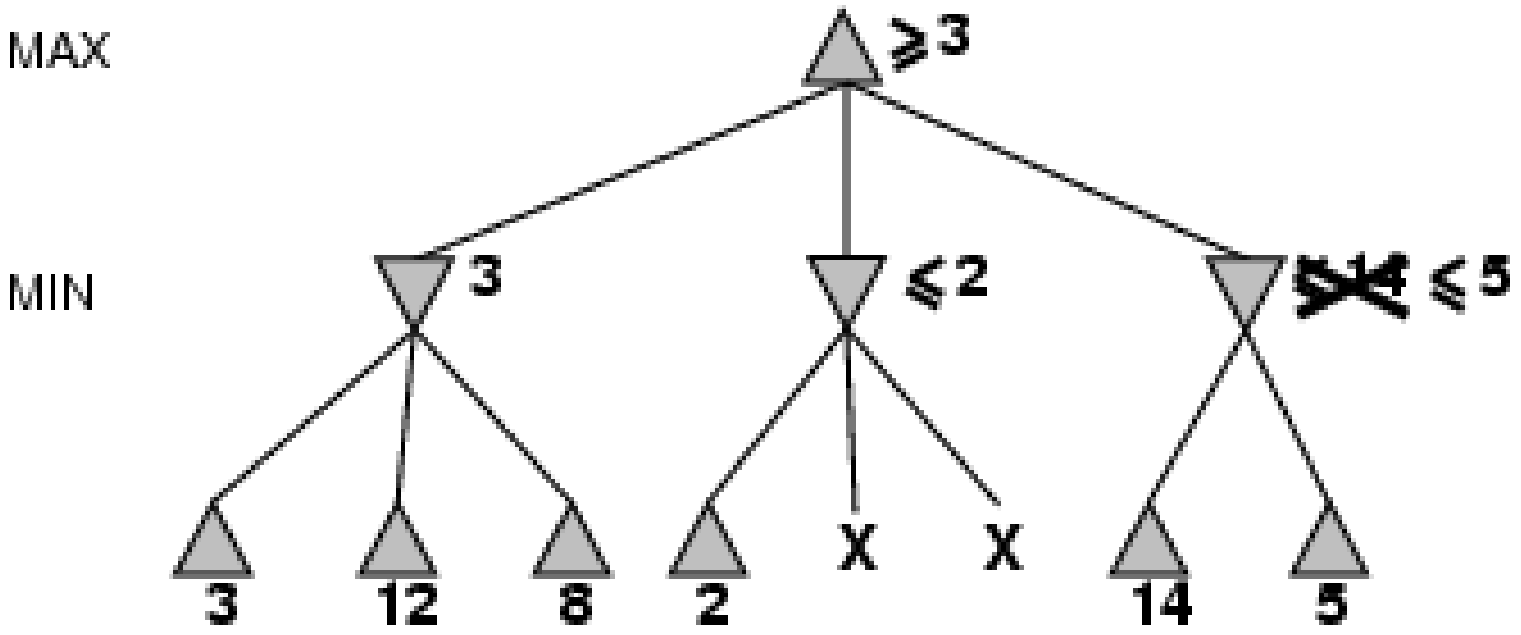
α-β pruning example



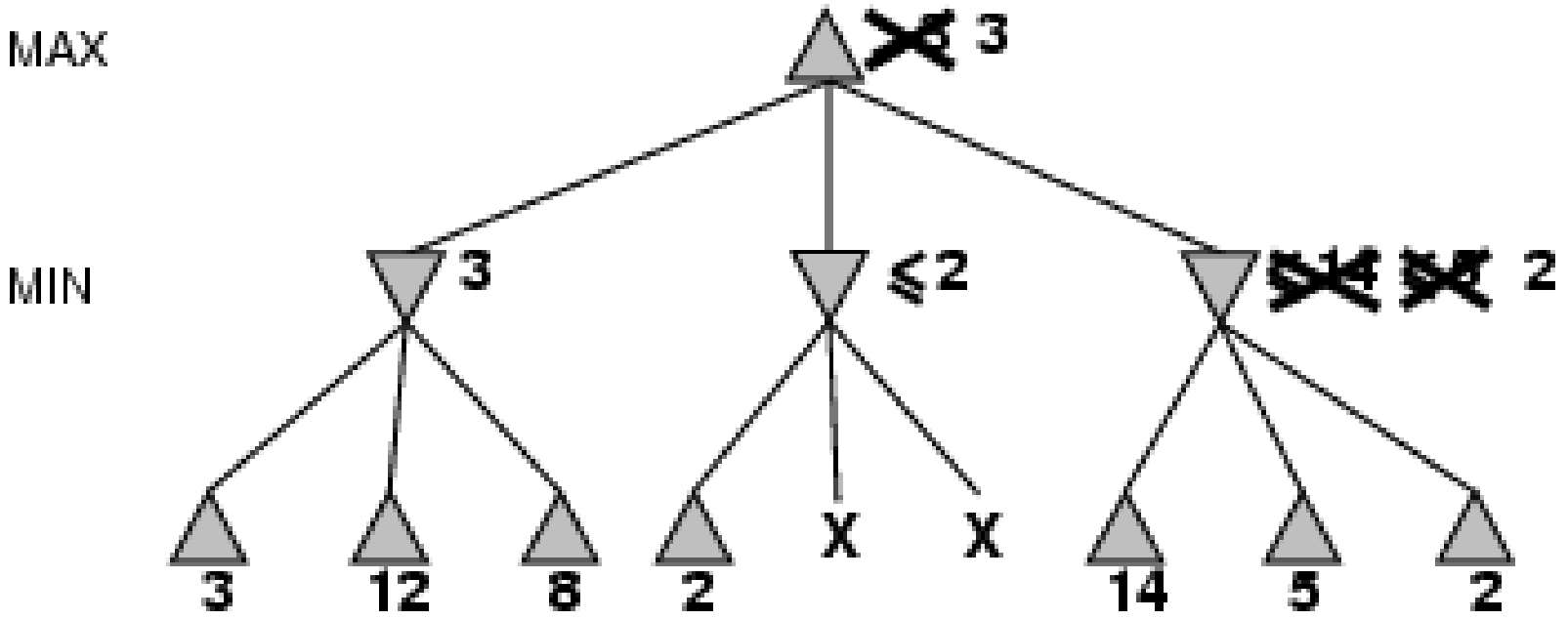
α-β pruning example



α-β pruning example

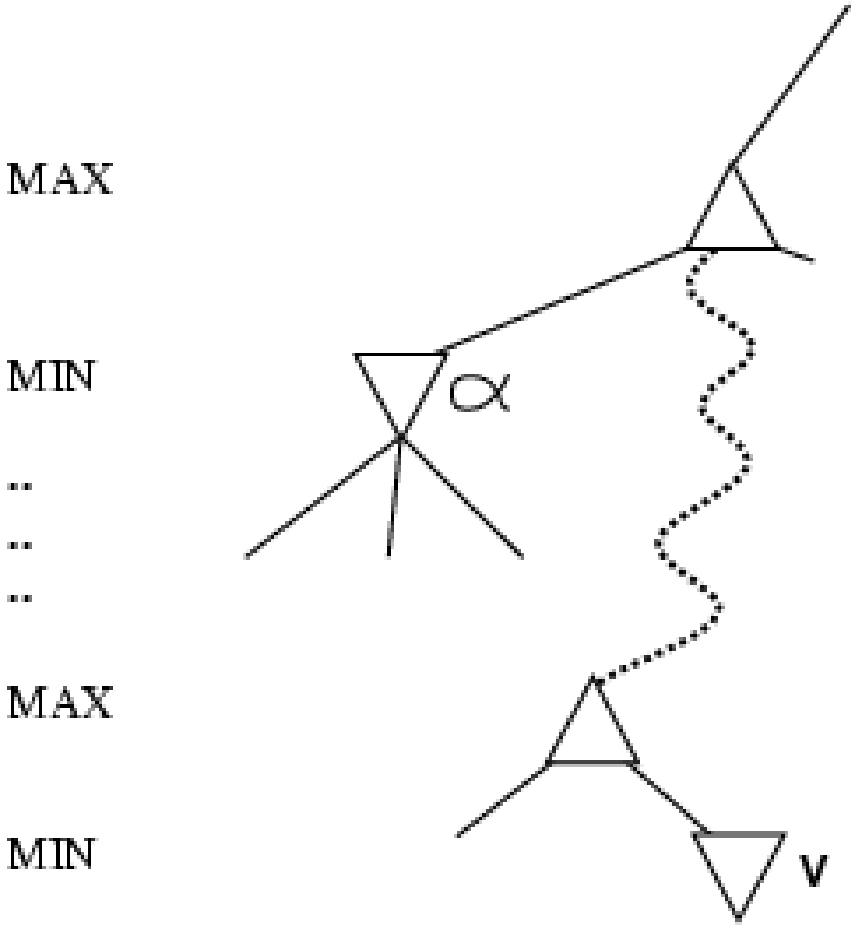


α-β pruning example



Why is it called α-β?

- ***α*** is the best value (to MAX) found so far off the current path
- If ***V*** is worse than **α**, MAX will avoid it
  - → prune that branch
  -
- Define **β** similarly for MIN



The α-β algorithm



https://www.youtube.com/watch?v=xBXHtz4Gbdo

The α-β algorithm

```
function ALPHA-BETA-SEARCH(state) returns an action
    inputs: state, current state in game

    v ← MAX-VALUE(state, −∞, +∞)
    return the action in SUCCESSORS(state) with value v

---

function MAX-VALUE(state, α, β) returns a utility value
    inputs: state, current state in game
            α, the value of the best alternative for MAX along the path to state
            β, the value of the best alternative for MIN along the path to state

    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do
        v ← MAX(v, MIN-VALUE(s, α, β))
        if v ≥ β then return v
        α ← MAX(α, v)
    return v
```

The α-β algorithm

```
function MIN-VALUE(state, α, β) returns a utility value
    inputs: state, current state in game
            α, the value of the best alternative for MAX along the path to state
            β, the value of the best alternative for MIN along the path to state

    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← +∞
    for a, s in SUCCESSORS(state) do
        v ← MIN(v, MAX-VALUE(s, α, β))
        if v ≤ α then return v
        β ← MIN(β, v)
    return v
```

The α-β algorithm

```
function ALPHA-BETA-DECISION(state) returns an action
    return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))

function MAX-VALUE(state, α, β) returns a utility value
    inputs: state, current state in game
            α, the value of the best alternative for MAX along the path to state
            β, the value of the best alternative for MIN along the path to state

    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do
        v ← MAX(v, MIN-VALUE(s, α, β))
        if v ≥ β then return v
        α ← MAX(α, v)
    return v

function MIN-VALUE(state, α, β) returns a utility value
    same as MAX-VALUE but with roles of α, β reversed
```

The α-β algorithm

```
function ALPHA-BETA-DECISION(state) returns an action
    return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))

function MAX-VALUE(state, α, β) returns a utility value
    inputs: state, current state in game
            α, the value of the best alternative for MAX along the path to state
            β, the value of the best alternative for MIN along the path to state

    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do
        v ← MAX(v, MIN-VALUE(s, α, β))
        if v ≥ β then return v
        α ← MAX(α, v)
    return v

function MIN-VALUE(state, α, β) returns a utility value
    same as MAX-VALUE but with roles of α, β reversed
```

Properties of α-β

Pruning does not affect final result

Good move ordering improves effectiveness of pruning

With "perfect ordering," time complexity = $O(b^{m/2})$

→ doubles depth of search

A simple example of the value of reasoning about which computations are relevant (a form of metareasoning)

Unfortunately, 35^{50} is still impossible!

Resource limits

Suppose we have 100 secs, explore 10^4 nodes/sec

→ 10^6 nodes per move

Standard approach:

cutoff test:

e.g., depth limit (perhaps add quiescence search)

evaluation function

= estimated desirability of position

Evaluation functions

For chess, typically linear weighted sum of features

$$\mathit{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

e.g., $w_1 = 9$ with

$f_1(s)$ = (number of white queens) – (number of black queens), etc.

Cutting off search

MinimaxCutoff is identical to *MinimaxValue* except

1. *Terminal?* is replaced by *Cutoff?*
2. *Utility* is replaced by *Eval*

Does it work in practice?

$b^m = 10^6$, b=35 → m=4

4-ply lookahead is a hopeless chess player!

4-ply ≈ human novice
8-ply ≈ typical PC, human master
12-ply ≈ Deep Blue, Kasparov

Deterministic games in practice

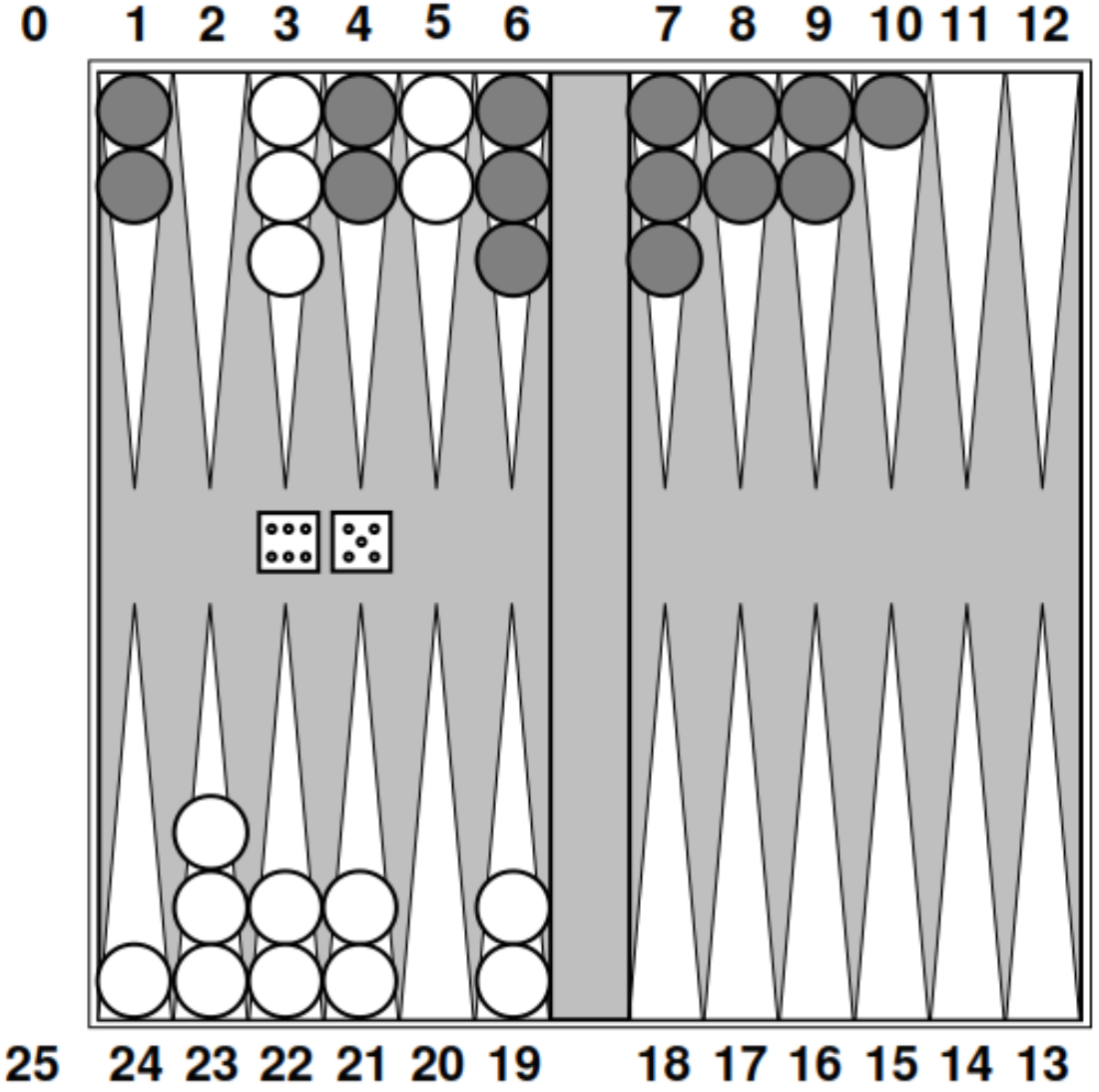
Checkers: Chinook ended 40-year-reign of human world champion Marion Tinsley in 1994. Used a precomputed endgame database defining perfect play for all positions involving 8 or fewer pieces on the board, a total of 444 billion positions.

Chess: Deep Blue defeated human world champion Garry Kasparov in a six-game match in 1997. Deep Blue searches 200 million positions per second, uses very sophisticated evaluation, and undisclosed methods for extending some lines of search up to 40 ply. $b^{m/2}$

Othello: human champions refuse to compete against computers, who are too good.

Go: human champions refuse to compete against computers, who are too bad. In go, $b > 300$, so most programs use pattern knowledge bases to suggest plausible moves.

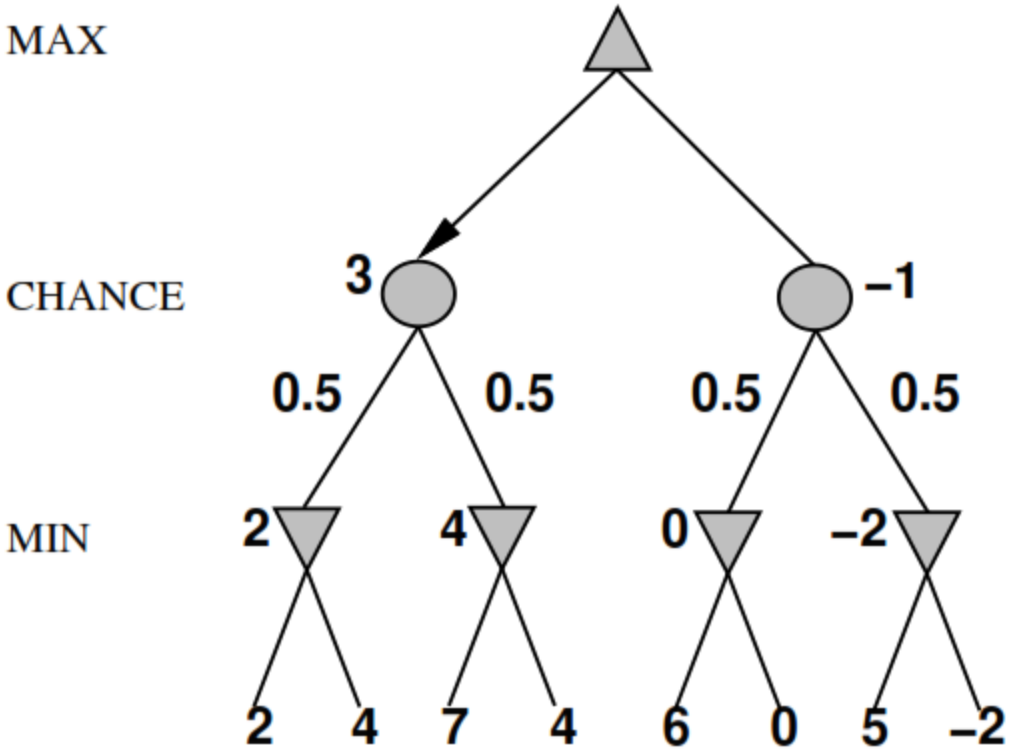
Nondeterministic games: backgammon



Nondeterministic games in general

In nondeterministic games, chance introduced by dice, card-shuffling

Simplified example with coin-flipping:



Summary

Games are fun to work on! (and dangerous)

They illustrate several important points about AI

- perfection is unattainable → must approximate
- good idea to think about what to think about
- uncertainty constrains the assignment of values to states
- optimal decisions depend on information state, not real state

Games are to AI as grand prix racing is to automobile design

Homework: *"Tic-Tac-Toe"*

Implement a game of Tic-Tac-Toe against an opponent using the Min-Max algorithm with Alpha-Beta Pruning.

It starts with a blank board.

At each step:

- One player enters two numbers ([1,3]) from a keyboard which are his or her turn on the board;
- Using the algorithm, we find our move;
- Then the specific status of the game is displayed.

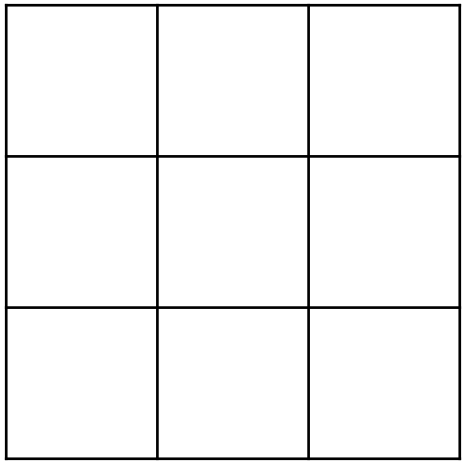
Finally, it shows who won the game.

** Make the game so that it can be set whether the computer or player is first.*

*** Make the algorithm optimal.*

"Tic-Tac-Toe" Optimality?

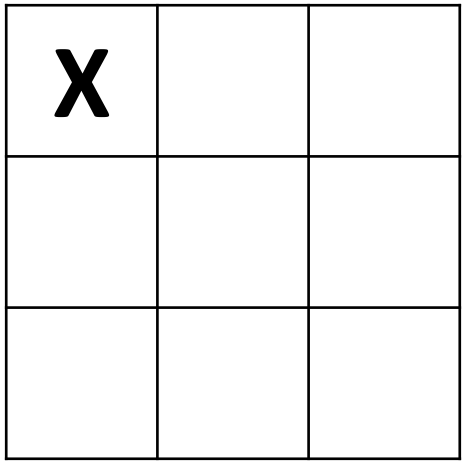
"Tic-Tac-Toe" Optimality?



X – Player *(First)*

O – Computer *(Second)*

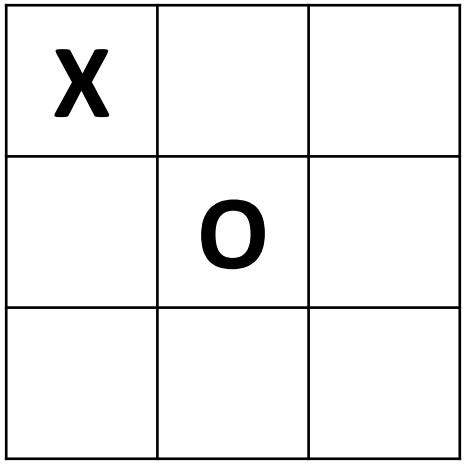
"Tic-Tac-Toe" Optimality?



X – Player *(First)*

O – Computer *(Second)*

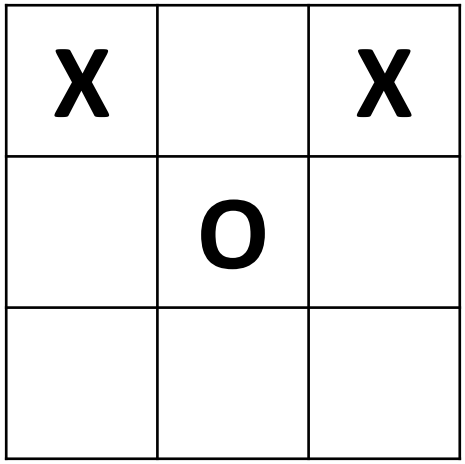
"Tic-Tac-Toe" Optimality?



X – Player *(First)*

O – Computer *(Second)*

"Tic-Tac-Toe" Optimality?



X – Player *(First)*

O – Computer *(Second)*

"Tic-Tac-Toe" Optimality?

X	**O**	**X**
	O	

X – Player *(First)*

O – Computer *(Second)*

"Tic-Tac-Toe" Optimality?

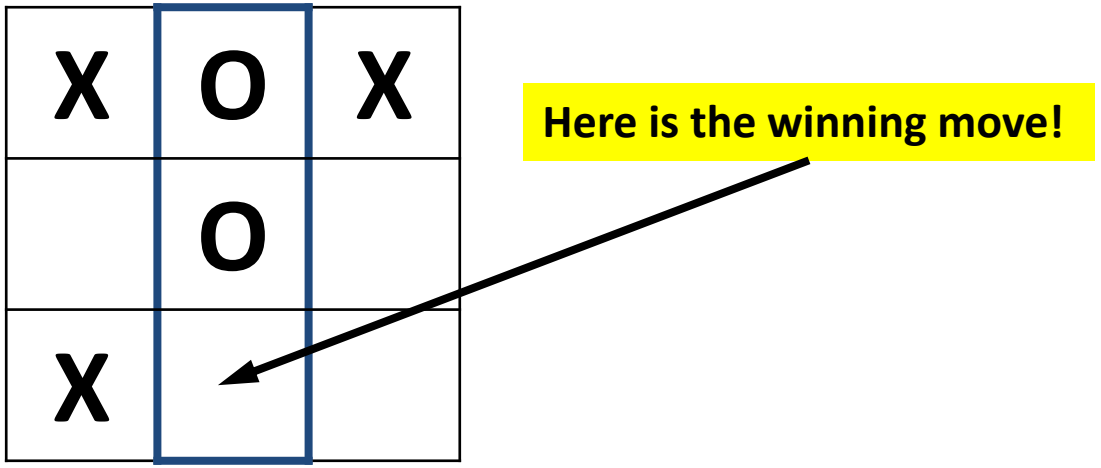
X	O	X
	O	
X		

The player is expected to block the computer, but instead plays a wrong move!

X – Player *(First)*

O – Computer *(Second)*

"Tic-Tac-Toe" Optimality?



X – Player *(First)*

O – Computer *(Second)*

"Tic-Tac-Toe" Optimality?

X	O	X
O	O	
X		

Why?

X – Player *(First)*

O – Computer *(Second)*

"Tic-Tac-Toe" Optimality?

X	O	X
O	O	X
X		

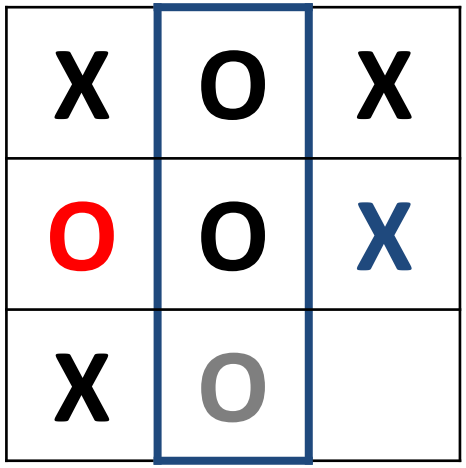
X	O	X
O	O	
X	X	

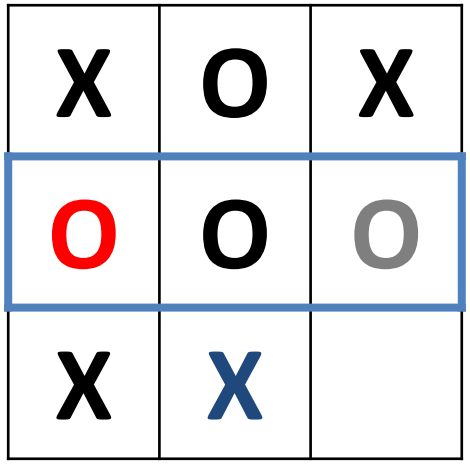
X	O	X
O	O	
X		X

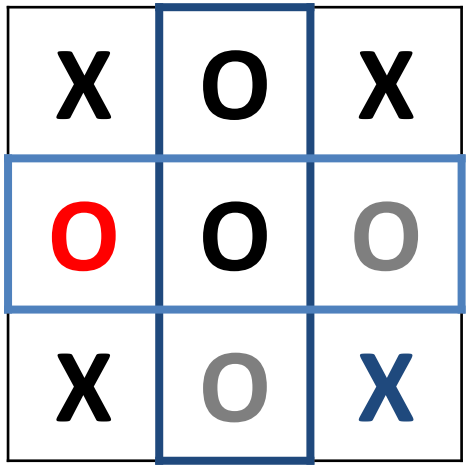
X – Player *(First)*

O – Computer *(Second)*

"Tic-Tac-Toe" Optimality?



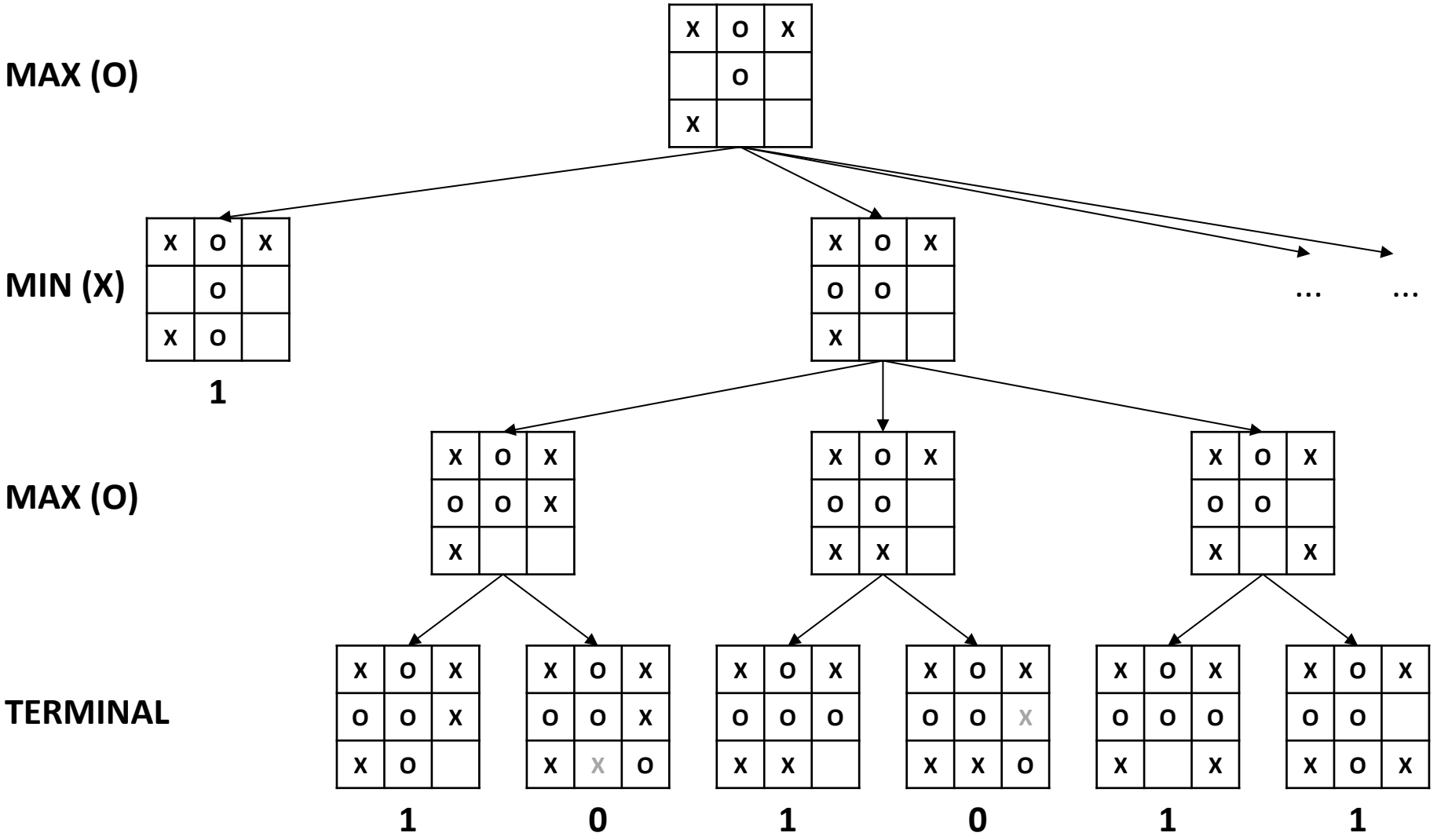




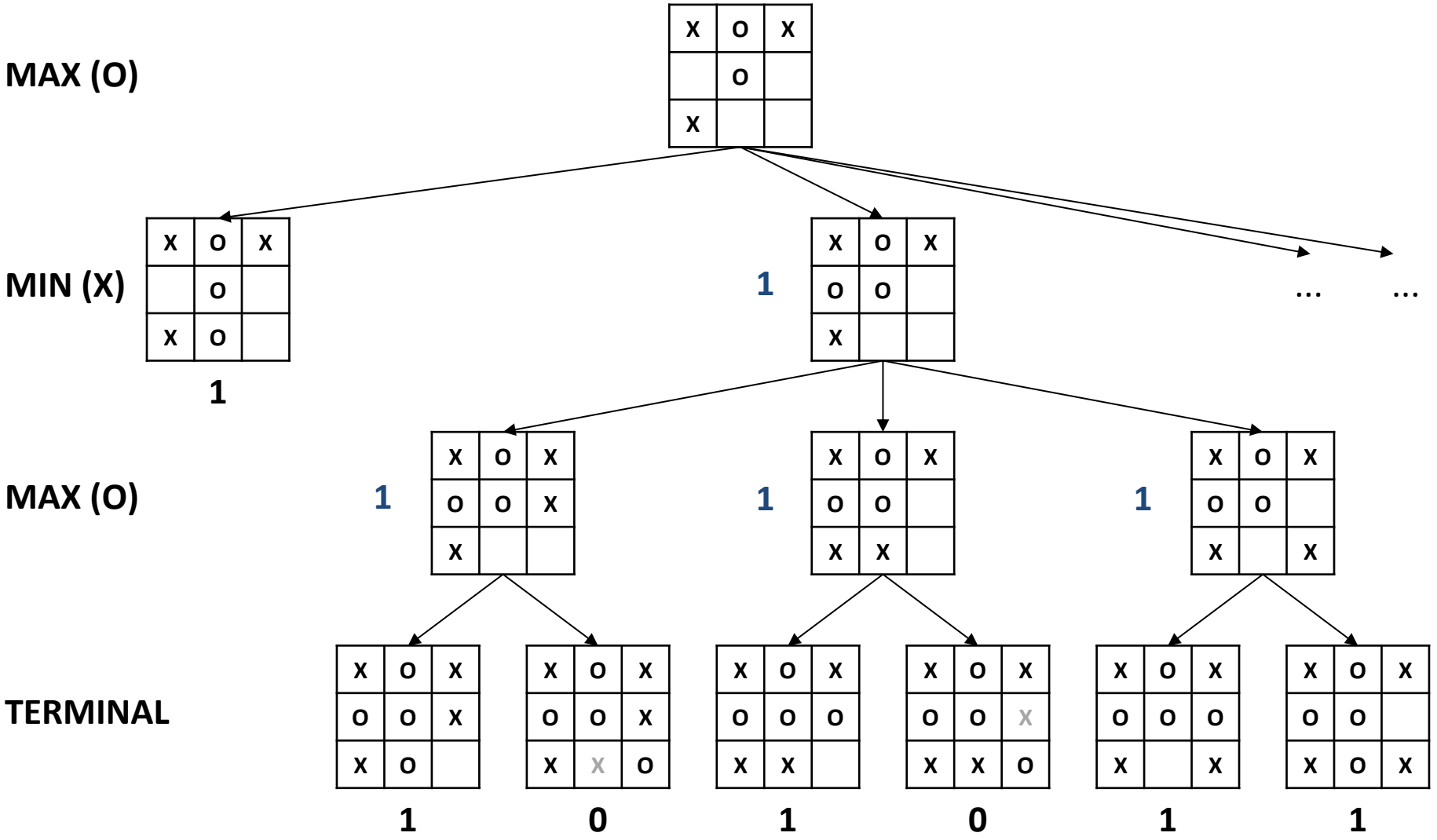
X – Player *(First)*

O – Computer *(Second)*

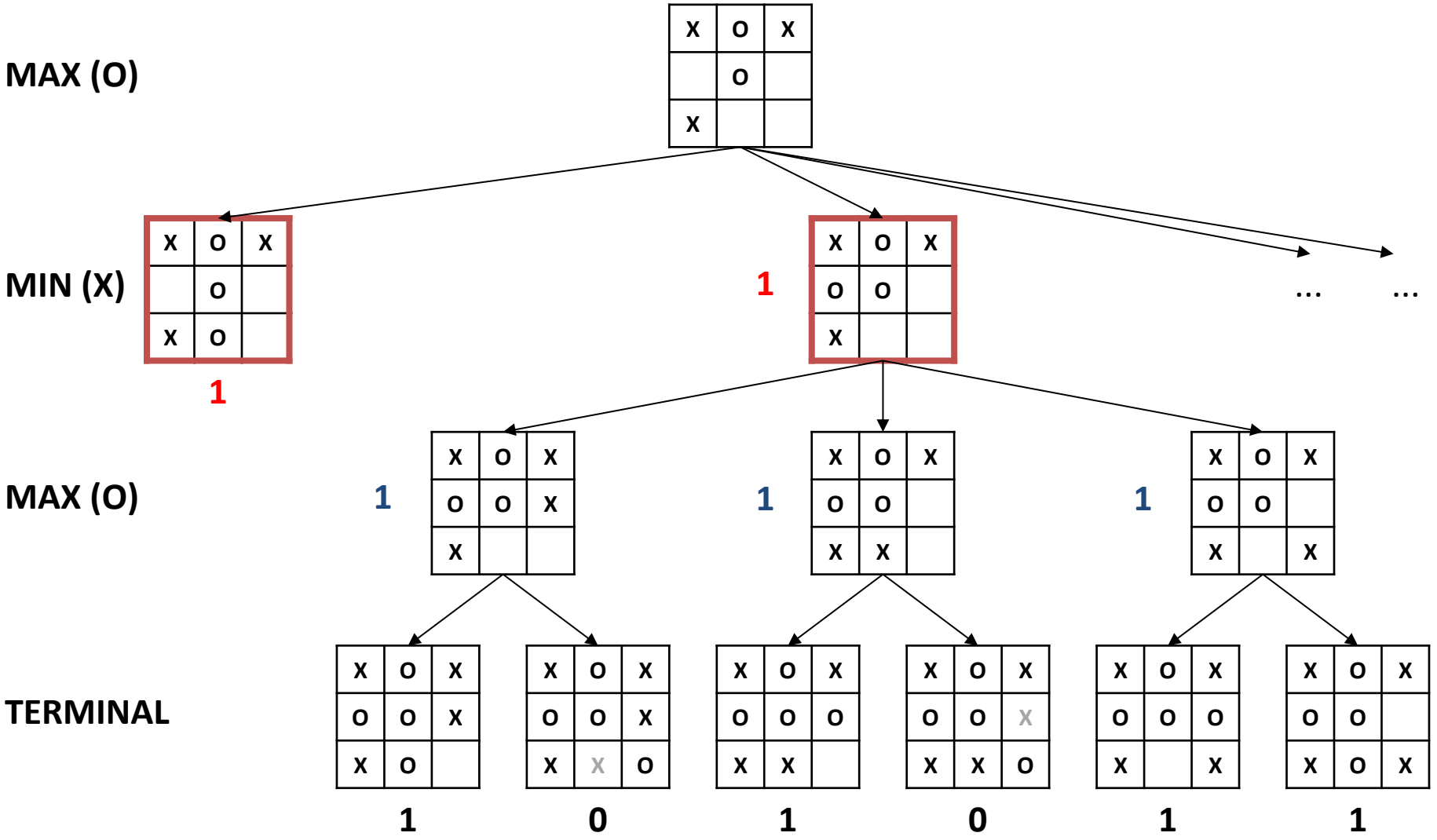
"Tic-Tac-Toe" Optimality?



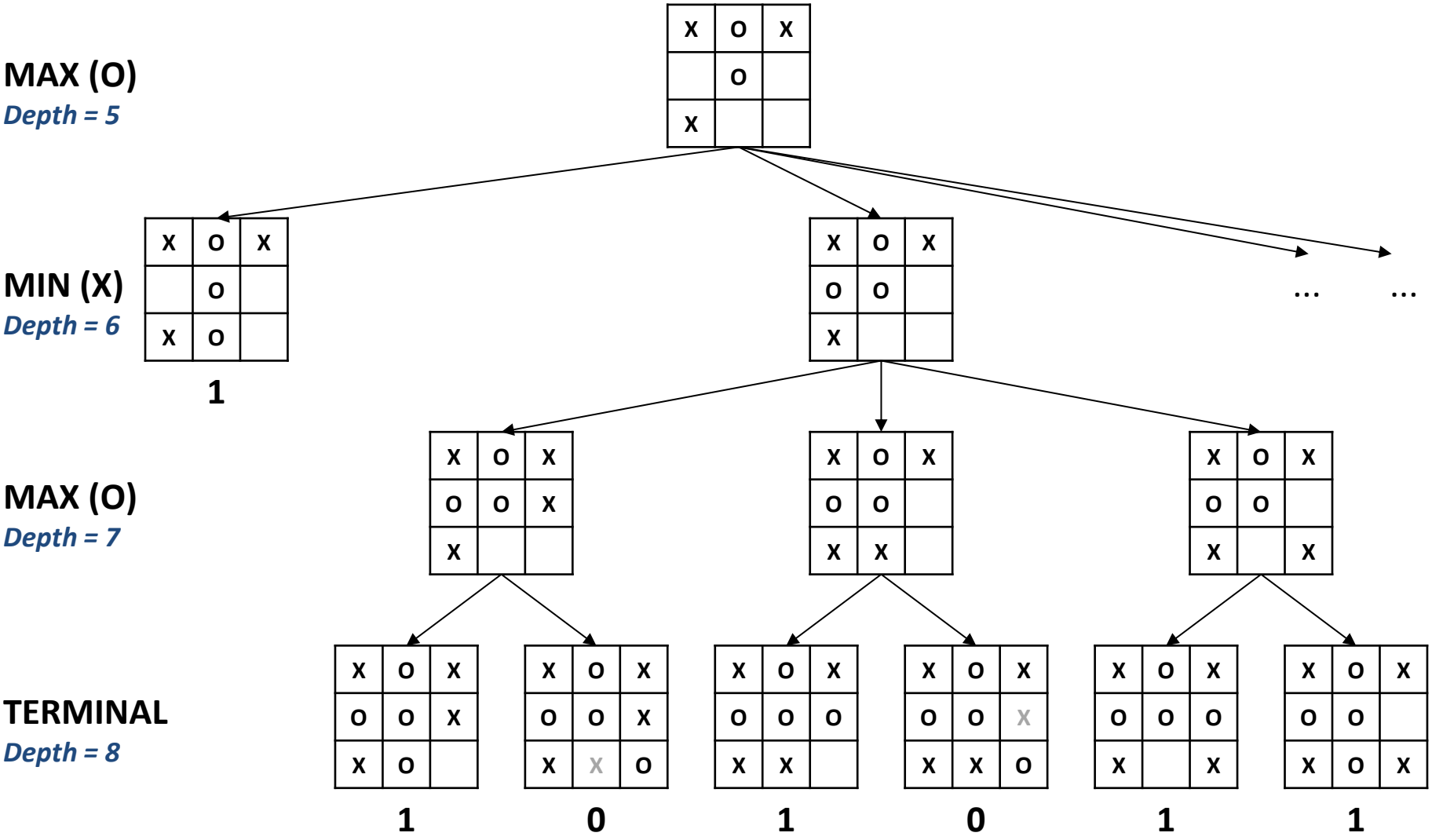
"Tic-Tac-Toe" Optimality?



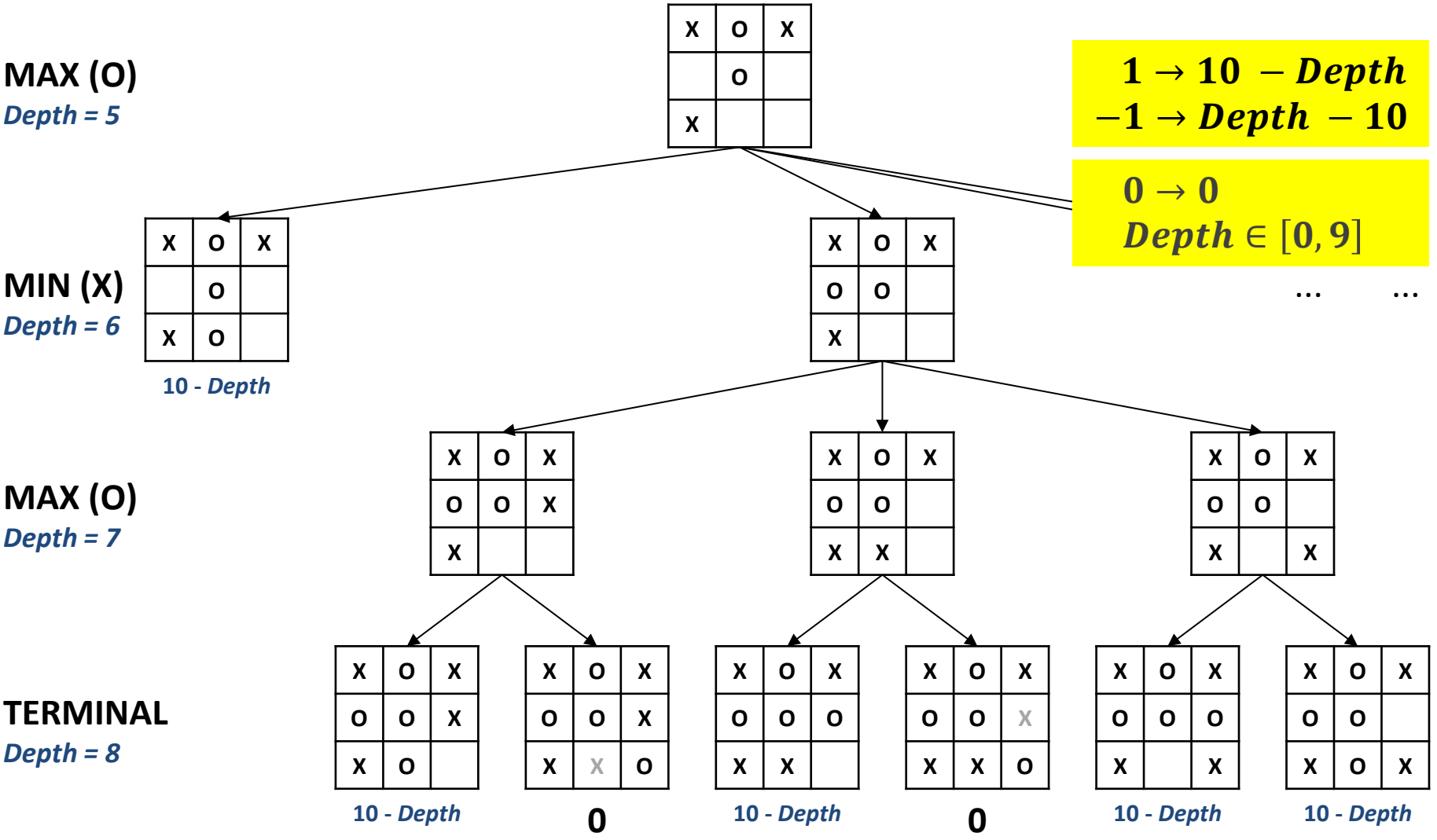
"Tic-Tac-Toe" Optimality?



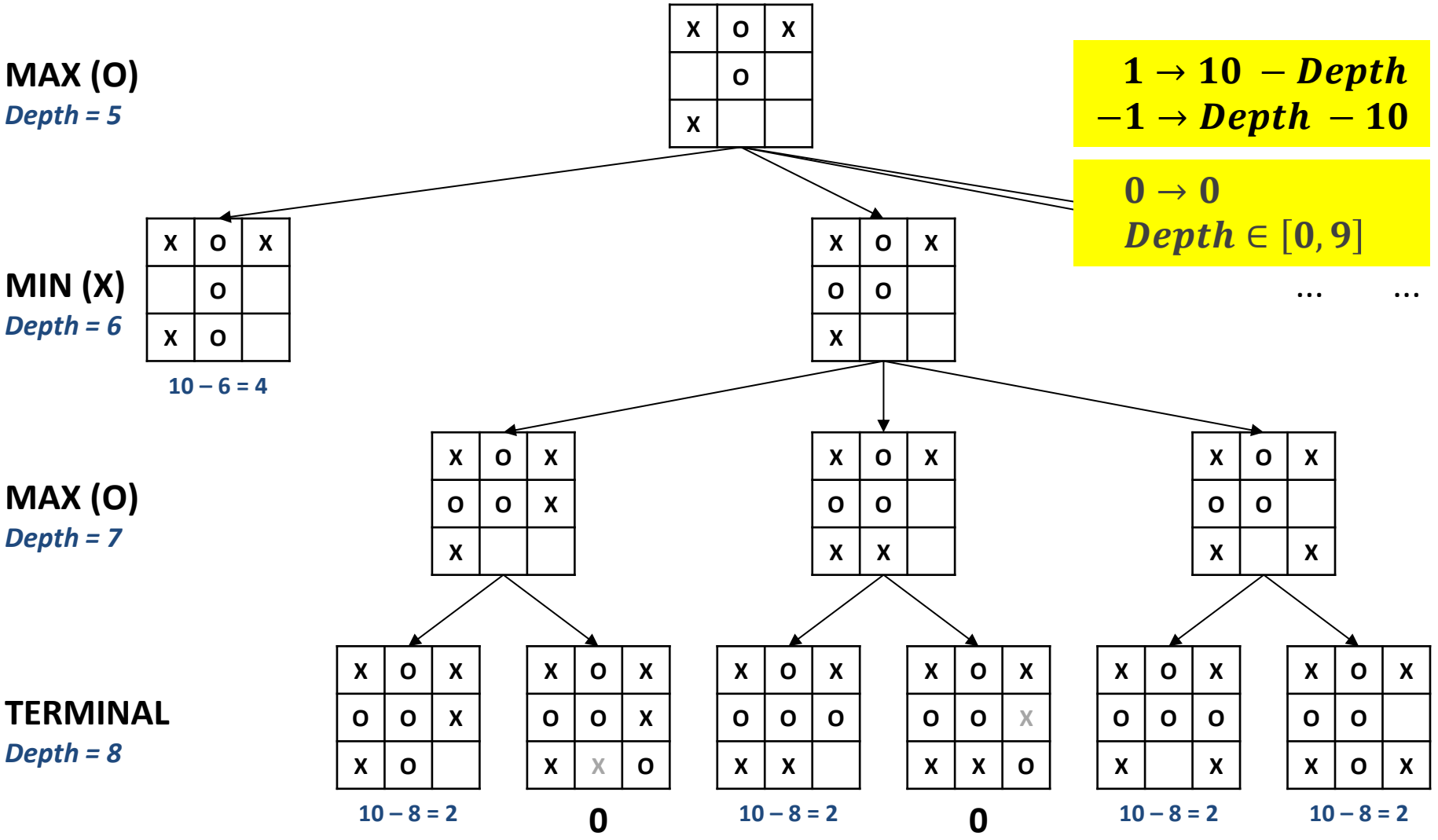
"Tic-Tac-Toe" Optimality?



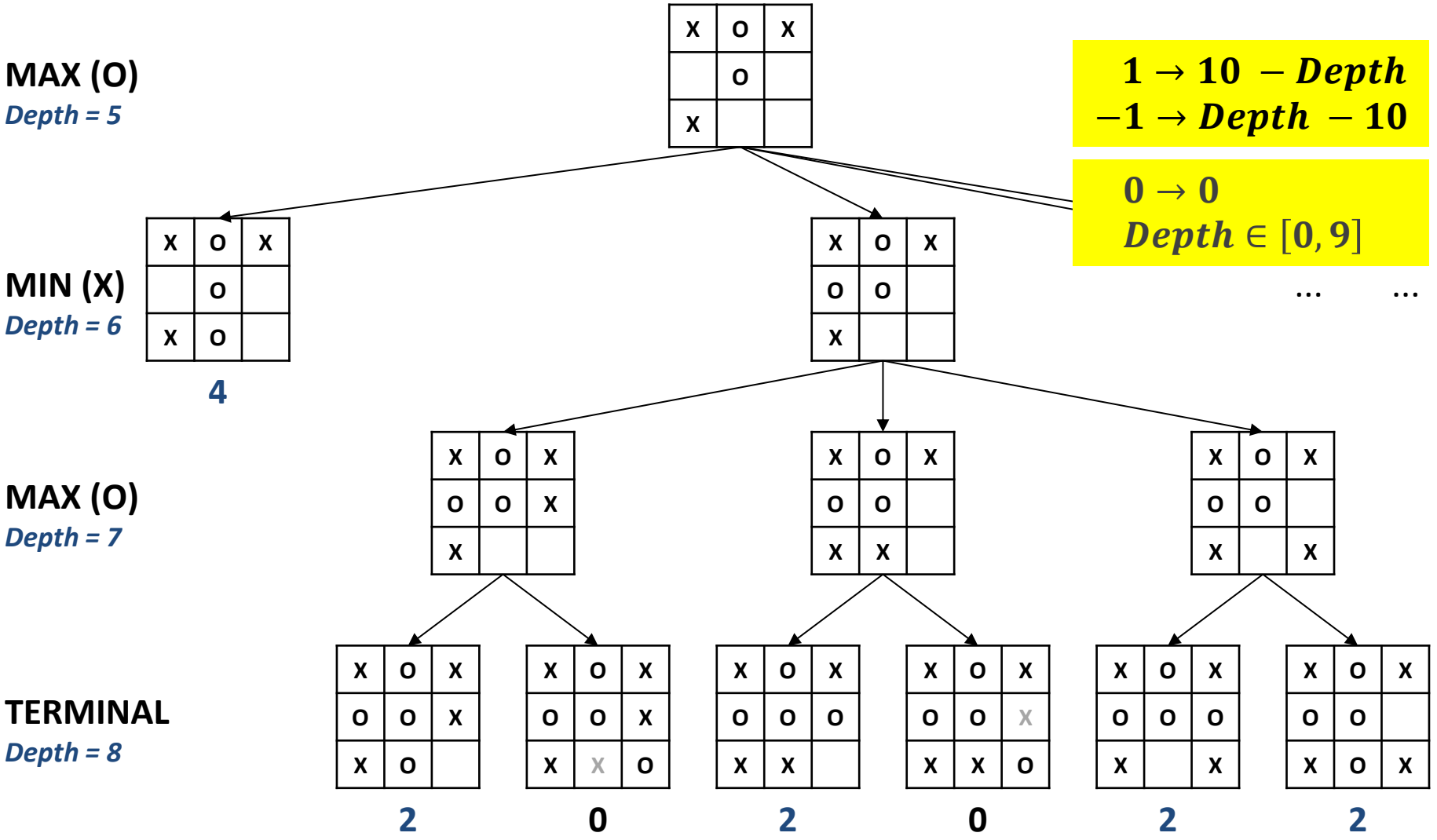
"*Tic-Tac-Toe*" Optimality?



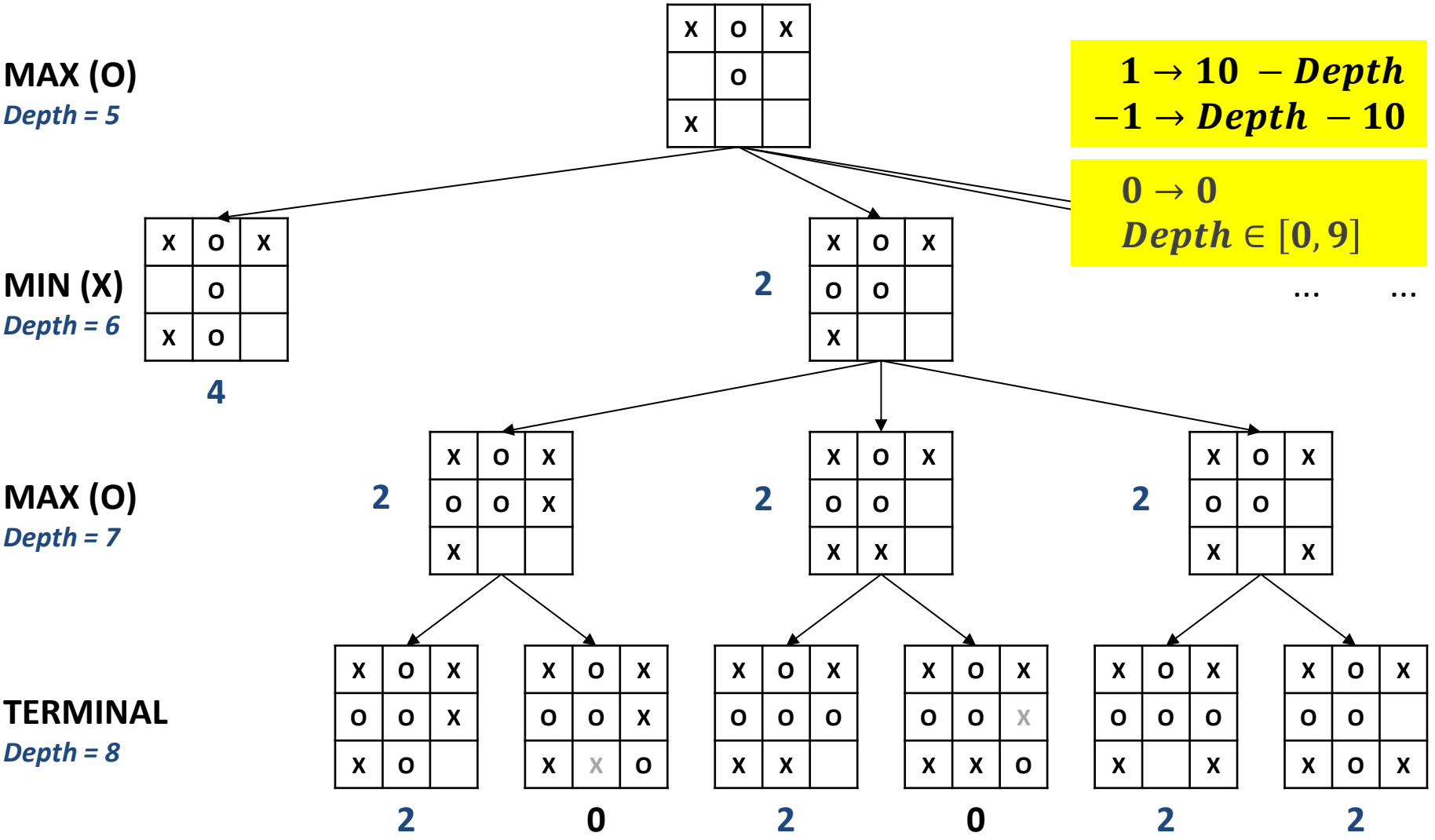
"*Tic-Tac-Toe*" Optimality?



"Tic-Tac-Toe" Optimality?



"*Tic-Tac-Toe*" Optimality?



"*Tic-Tac-Toe*" Optimality?

